MILESTONE 1

Topic : Entertainment Domain

Prajwal

4SF24CS139

CSE - 4A

SECTION A: — SCHEMA ANALYSIS AND NORMALIZATION

Given Schema:-

Bookings (
  booking_id,
  user_name,
  user_email,
  movie_name,
  theatre_name,
  theatre_location,
  show_time,
  seat_numbers,
  total_price,
  payment_status
)

- **A1. Issues with reasons**

    1. **Data redundancy- Theatre location ,theatre name and movie name etc.., will be redundant in the given schema .**

    2. **The theatre cannot be added without booking ticket.**

3. **The given table is not in 2NF- because there are many columns with partial dependency.**

4. **The given table is not in 3NF- because there are columns with transitive dependency.**

- ## A2. Normalize the Schema

1NF: one user can book multiple seats at a time ,if we store all those seats in same row then it violates 1nf (because atomic values).there are 2 solutions either we can store all the seats in different row(redundancy will be the problem) or we can create 2 separate table(best because no redundancy) one for bookings and another for seats.

Bookings (
booking_id(pk),
user_name,
user_email,
movie_name,
theatre_name,
theatre_location,
show_time,
total_price,
payment_status
)

Seats(
Seat_id(pk),
Screen_id(fk),
Seat_no ,
Seat_type
);

Missing attributes

Bookings (

booking_id(PK),

user_id,

user_name,

user_email,

movie_id,

movie_name,

theatre_id,

theatre_name,

theatre_location,

screen_no,

show_id,

show_time,

total_price,

payment_status,

booking_status,

payment_id,

payment_time

)

2NF:

The schema in the 1NF is already in the 2NF because using booking_id we can find all the other values which means all the other values are dependent on booking_id.

3NF:

There are many attributes with transive dependency for example –

Username and email are dependent on userid and theatre name and locations are dependent on theatreid et.., so dividing the above entity gives the following tables.

```
CREATE TABLE Users (
   user_id INTEGER PRIMARY KEY,
   user_name VARCHAR(100),
   user_email VARCHAR(100) UNIQUE
);

CREATE TABLE Movies (
   movie_id INTEGER PRIMARY KEY,
   movie_name VARCHAR(100)
);

CREATE TABLE Theatres (
   theatre_id INTEGER PRIMARY KEY,
   theatre_name VARCHAR(100),
   theatre_location VARCHAR(100)
);

CREATE TABLE Screens (
   screen_id INTEGER PRIMARY KEY,
   theatre_id INT,
   screen_no INT,
   FOREIGN KEY (theatre_id) REFERENCES Theatres(theatre_id),
   UNIQUE (theatre_id, screen_no)
);
CREATE TABLE Shows (
   show_id INTEGER PRIMARY KEY,
   movie_id INT,
   screen_id INT,
   show_time DATETIME,
```

```
    FOREIGN KEY (movie_id) REFERENCES Movies(movie_id),
    FOREIGN KEY (screen_id) REFERENCES Screens(screen_id)
);

CREATE TABLE Seats (
    seat_id INTEGER PRIMARY KEY,
    screen_id INT,
    seat_no VARCHAR(10),
    seat_type VARCHAR(20),
    FOREIGN KEY (screen_id) REFERENCES Screens(screen_id),
    UNIQUE (screen_id, seat_no)
);

CREATE TABLE Payments (
    payment_id INTEGER PRIMARY KEY,
    payment_status VARCHAR(20),
    payment_time DATETIME
);

CREATE TABLE Bookings (
    booking_id INTEGER PRIMARY KEY,
    user_id INT,
    show_id INT,
    payment_id INT,
    booking_status VARCHAR(20),
    total_price DECIMAL(10,2),
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (show_id) REFERENCES Shows(show_id),
    FOREIGN KEY (payment_id) REFERENCES Payments(payment_id)
);
CREATE TABLE Booked_Seats (
    show_id INT,
```

seat_id **INT**,

booking_id **INT**,

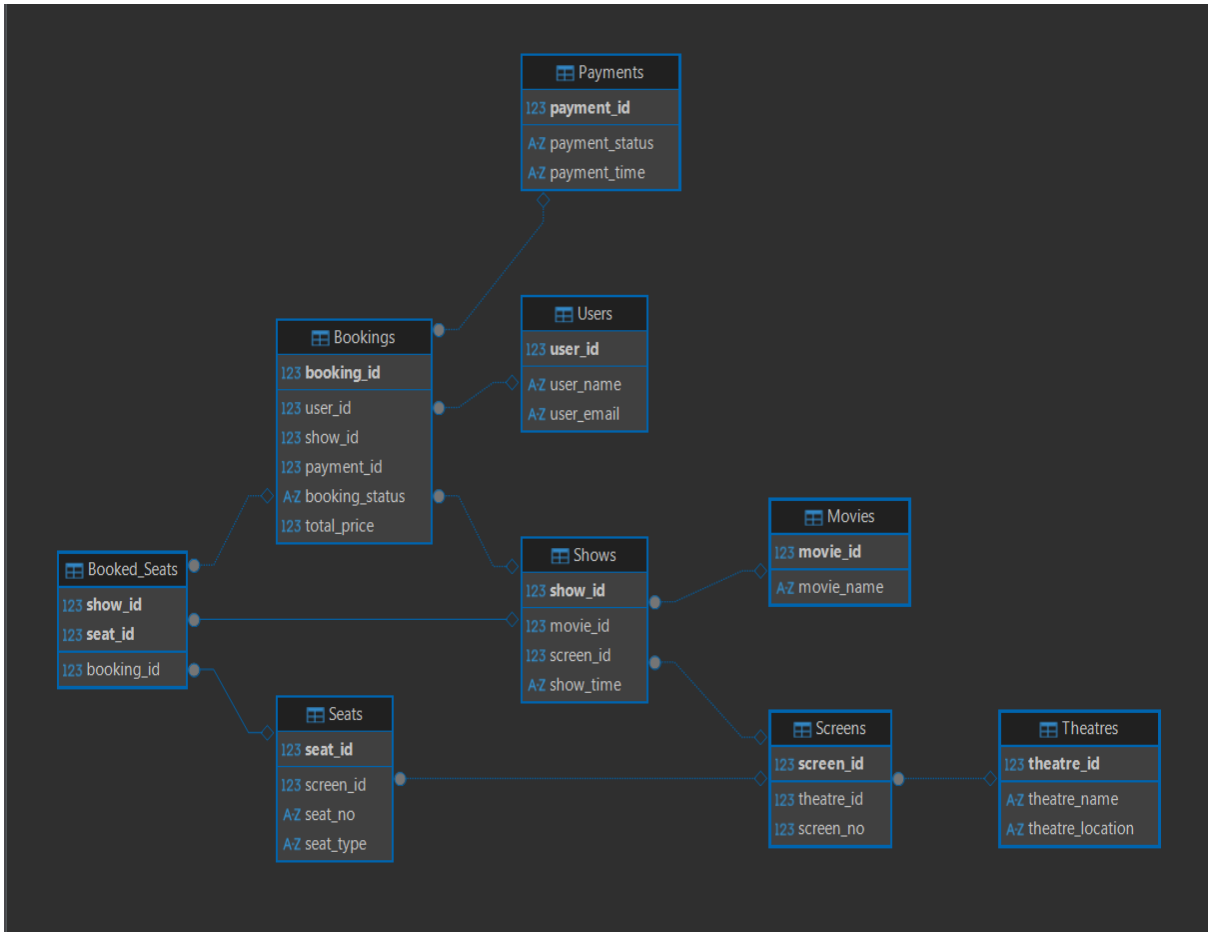
PRIMARY KEY (show_id, seat_id),

FOREIGN KEY (show_id) **REFERENCES** Shows(show_id),

FOREIGN KEY (seat_id) **REFERENCES** Seats(seat_id),

FOREIGN KEY (booking_id) **REFERENCES** Bookings(booking_id)

);



A3. Assumptions:

- We assume that the one booking can have many seats.
- The one booking can have only one user and we don't store the user for each seats ,we only store the user who wants to book seats.
- The one user can have many bookings.
- Multiple screens can have same seat numbers.

- Multiple theatres can have same screen numbers.
- The same screens can be used for multiple shows.
- The same seats can be booked for multiple shows.

SECTION B — DATABASE DESIGN FROM REQUIREMENTS

B1. Requirement Clarification:

- Can one booking have many seats?
- Should we store user details per seats?
- Can user have many bookings?
- Can multiple screens have same seat numbers?
- Can multiple theatres have same screen numbers?
- Can same screens be used for multiple shows?
- Can same seats be booked for multiple shows?
- Can seat price differ per shows?

B2. Design & B3. Normalization:

Users(
u_id (PK),
u_name,
u_email
)

Movies(
movie_id (PK),
movie_name
)

Theatre(
t_id (PK),
t_name,
t_location
)

Screens(
screen_id (PK),
screen_no,
theatre_id (FK)
)

Show(
show_id (PK),
show_time,
screen_id (FK),
movie_id (FK)
)

Seat_Type(
  type_id (PK),
  seat_type
)

Seats(
  seat_id (PK),
  seat_no,
  screen_id (FK),
  type_id (FK)
)

Bookings(
  book_id (PK),
  book_status,
  total_price,
  u_id (FK),
  show_id (FK),
  p_id (FK)
)

BookedSeats(
  seat_id (FK),
  show_id (FK),
  book_id (FK),
  PRIMARY KEY (show_id, seat_id)
)

Payment(
  p_id (PK),
  status,
  time
)

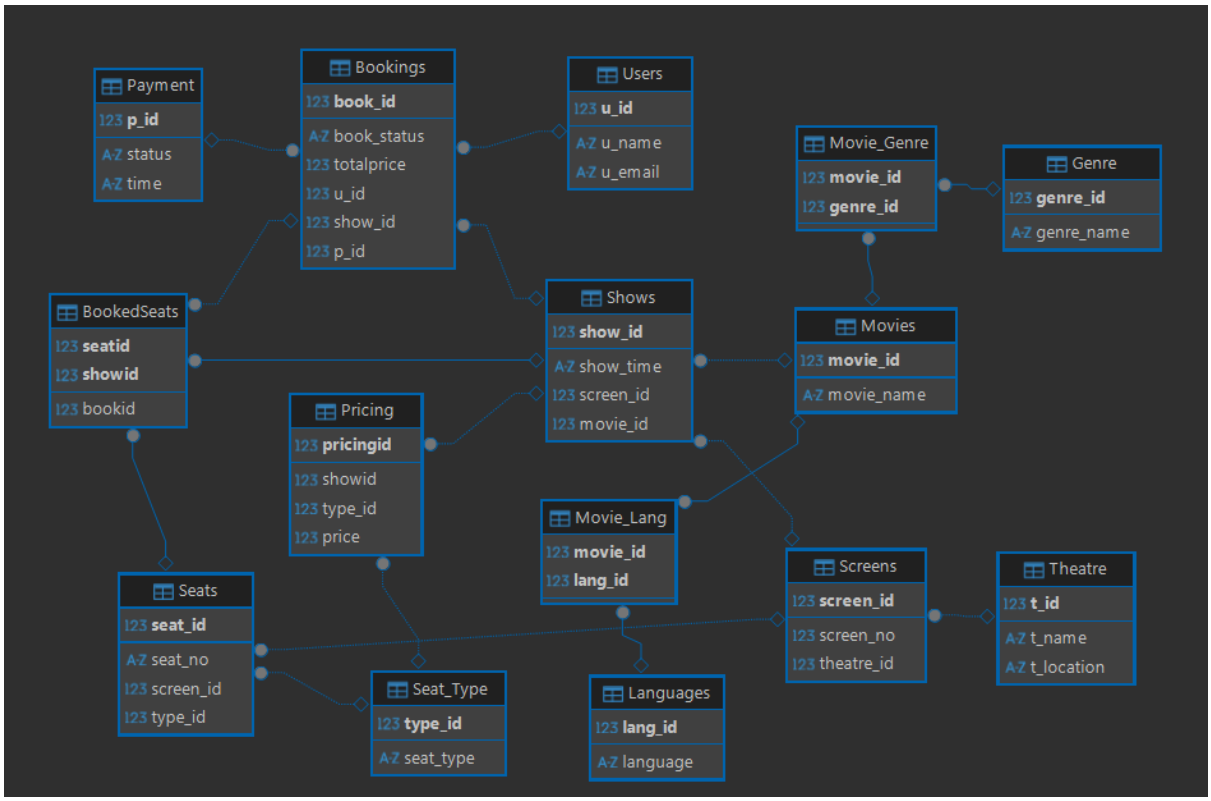
Pricing(
  pricing_id (PK),
  show_id (FK),
  type_id (FK),
  price
)

Languages(
  lang_id (PK),
  language
)

Movie_Lang(
  movie_id (FK),
  lang_id (FK),
  PRIMARY KEY (movie_id, lang_id)
)

Genre(
  genre_id (PK),
  genre_name
)

Movie_Genre(
  movie_id (FK),
  genre_id (FK),
  PRIMARY KEY (movie_id, genre_id)
)



Update:

- A single booking can include multiple users
  =Our design now handles one user can have multiple bookings but not one booking multiple users. Now our design is many to one or one to many to make it many to many we have to add another table .

  Booking_Users(
    booking_id,
    user_id,
    PRIMARY KEY (booking_id, user_id)

)

And update by removing u_id

Bookings(
  book_id (PK),
  book_status,
  total_price,
  show_id (FK),
  p_id (FK)
)

- Each user must be individually associated with the seats booked.
  = For this update the following table

BookedSeats(
  seat_id (FK),
  show_id (FK),
  book_id (FK),
  user_id (FK),
  PRIMARY KEY (show_id, seat_id)
)

SECTION C — QUERIES AND TRANSACTIONS

- Retrieve all bookings for a given user within a specified date range, including movie name, theatre, show time, and seats booked.

```sql
SELECT
    b.book_id,
    m.movie_name,
    t.t_name,
    s.show_time,
    count(se.seat_no) as seats_booked
FROM Bookings b
JOIN Booking_Users bu ON b.book_id = bu.booking_id
JOIN Shows s ON b.show_id = s.show_id
JOIN Movies m ON s.movie_id = m.movie_id
JOIN Screens sc ON s.screen_id = sc.screen_id
JOIN Theatre t ON sc.theatre_id = t.t_id
JOIN BookedSeats bs ON b.book_id = bs.bookid
JOIN Seats se ON bs.seatid = se.seat_id
WHERE bu.user_id = 1
GROUP BY b.book_id;
```

Bookings(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

	book_id	movie_name	t_name	show_time	seats_booked
1	1	Inception	PVR	2026-05-01 10:00:00	2
2	3	Interstellar	PVR	2026-05-01 18:00:00	1

- Identify the most frequently booked movie along with total bookings count.

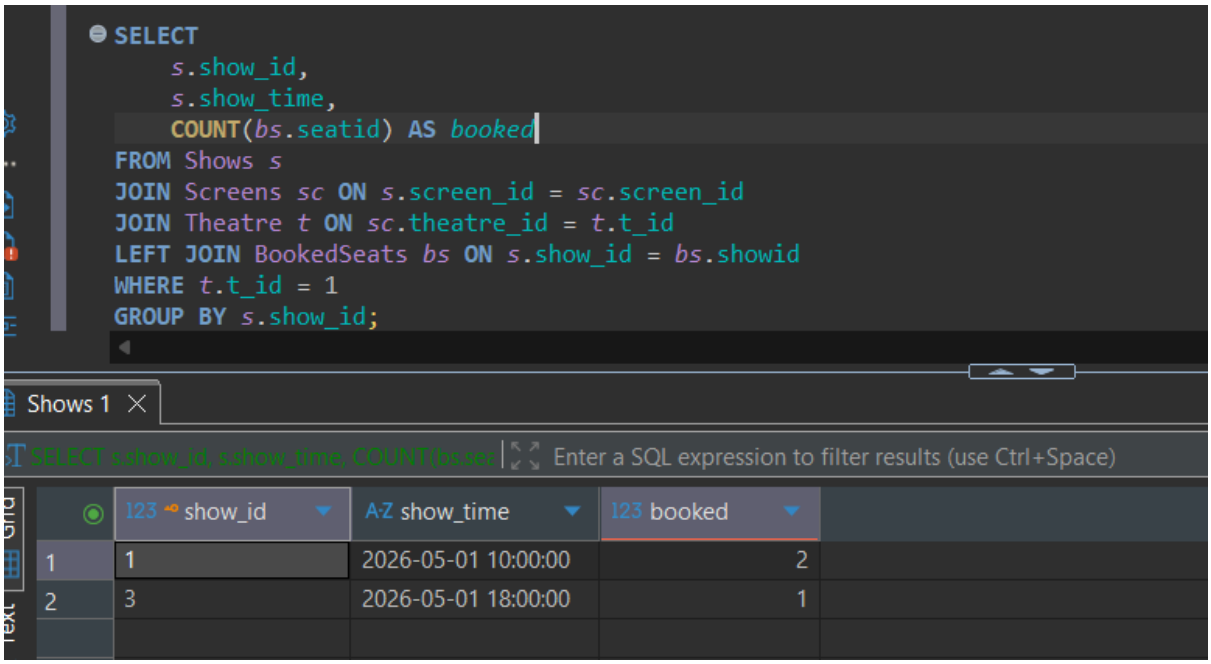
```sql
SELECT
    m.movie_name,
    COUNT(b.book_id) AS total
FROM Bookings b
JOIN Shows s ON b.show_id = s.show_id
JOIN Movies m ON s.movie_id = m.movie_id
GROUP BY m.movie_id
ORDER BY total DESC
LIMIT 1;
```

Movies 1

Enter a SQL expression to filter results (use Ctrl+Space)

	movie_name	total
1	Interstellar	1

- For a given theatre and date, list all shows along with total seats booked and available seats.



- Write a transaction to handle a booking operation:

```sql
BEGIN TRANSACTION;

SELECT *
FROM BookedSeats
WHERE showid = 1 AND seatid IN (1,2);


INSERT INTO Bookings
VALUES (101, 'Confirmed', 500, 1, 1);

INSERT INTO Booking_Users VALUES (101, 1);
INSERT INTO Booking_Users VALUES (101, 2);

INSERT INTO BookedSeats VALUES (1, 1, 101, 1);
INSERT INTO BookedSeats VALUES (1, 2, 101, 2);

COMMIT;
```